NYTHOS SECURITY · DIGITAL SECURITY SIMPLIFIED

```
        /\
       /  \
      / /\ \
     / /__\ \
    /__/  \__\
    \  \  /  /
     \  \/  /
      \    /
       \  /
        \/
```

AQUARIUM MODE

FIELD GUIDE & ASSESSMENT

```
<==(::::::: O>     <~~(::::°:·:°::::: O>~     ><(°>
   <<<::·:·:· o>      .-^-.      <=::::::::::::::·°>
                                  ( o o )
                                    )|(
```

VERSION 1.0.0 · SECURE · MONITOR · ANALYZE · DEFEND

What Aquarium Mode Is

Aquarium Mode is an immersive, cyberpunk ASCII aquarium that doubles as a live endpoint-health visualization for the Nythos platform. It runs as a splash screen, idle screensaver, background dashboard, demo, or conference kiosk. The reef is alive, and it is **driven by real telemetry** — the tank visibly reflects the health of the machine it runs on. The goal is a single, calm message: *"this security software feels alive."*

THE CORE IDEA

Every fish is an autonomous agent. Every visual maps to a real signal. A calm purple tank means a healthy endpoint; a darkening tank with a circling predator means risk is climbing. Nothing is random when live data is available.

Contents

02	Architecture at a glance — the engine & its modules
03	Anatomy of the tank — water, reef, particles, camera
04	The inhabitants — fish tiers, koi & angelfish, boids AI, fauna
05	Security visualization — states, predator, sonar, AI beam
06	The HUD & live telemetry
07	Run modes, controls & easter eggs
08	Install & build
09	How to properly assess it — evaluation guide & rubric

READING THIS GUIDE

Each section first **explains a part of the aquarium**, then the final section gives you a **repeatable way to assess** whether it is behaving correctly and looking right.

The Engine & Its Modules

A modular, mode-agnostic engine. The renderer knows nothing about "fish" specifically, so future scenes (Space, Forest, Data Center...) can be added by swapping the water-layer generator and the entity set — without rewriting the renderer.

MODULE	RESPONSIBILITY
`app · __main__`	CustomTkinter shell, run modes, key bindings, window/fullscreen handling.
`engine`	Frame-paced timing loop and smoothed FPS; orchestrates everything each frame.
`scene`	The world + the security-state machine; single source of truth.
`renderer`	Pillow water layer + pooled Canvas entity layer + vector effects.
`entities · fish`	Boids fish agents (cohesion / alignment / separation / flee).
`predators · fauna`	Shark threat predator + jellyfish, turtle, octopus, stingray, whale, shrimp.
`ascii_art · ascii_generator`	Sprite library + procedural fish (parts, colour, personality).
`decor`	Reef floor: coral, kelp, treasure, ruins, rocks.
`particles`	Bubbles + drifting motes (pooled, stochastic).
`camera`	Cinematic drift + breathing zoom.
`effects`	Sonar sweep, AI beam, glitch burst, threat alert pulse.
`telemetry`	Live `psutil` snapshot — the Nythos data feed seam.
`hud · splash`	Brand/status HUD + debug overlay; startup loading sequence.
`easter_eggs · audio`	Hidden key sequences + optional sonar pings.

FRAME PIPELINE

```
[telemetry thread]  psutil → Snapshot  ───────────────┐
[main 60 FPS loop]  camera → scene.update → renderer.draw → hud.draw → present
                                   │                       │
                         state machine           water layer (≤12 Hz)
                         + boids + effects       + pooled ASCII entities
```

> WHY IT STAYS SMOOTH
>
> The water layer is rendered small + blurred and regenerated at ~12 Hz; entity Canvas items are **pooled** (repositioned, never recreated); telemetry polling runs off the UI thread. Allocations stay flat for 24/7 operation.

Reading the Scene

From back to front, the tank is composed in layers. Every layer moves — nothing is static.

WATER LAYER

- **Depth gradient** — violet surface fading to abyssal black.
- **Volumetric light rays** drifting from the surface.
- **Fog pools** — soft violet glows that wander.
- **Threat tint** — the whole field darkens & reddens under risk.

REEF & DECOR

- **Coral** & swaying **kelp** along the floor.
- **Treasure chest** stamped with the Nythos N.
- **Ruins**, **rocks** framing the depths.
- Each item sways on its own phase — a living reef.

PARTICLES

- **Bubbles** rise & wobble from the floor.
- **Motes** drift in the current.
- Emission is stochastic — the field never visibly loops.

CAMERA

- Slow **Lissajous drift** + faint breathing zoom.
- Gives a hand-held, “alive” feel.
- Never disorienting; toggle in settings.

THE REEF FLOOR

```
)  (   \|/    .Y..Y.   ._____________.   \|/  )(
(  )  \\|//  (|||||)  /::::::::::::::/|   //|  (
)  (   \|/    \|||/   |####[###]####| |   \|   )
~~~~~~~~~~~~~~~~~~~~~  |##( N )######|/  ~~~~~~~
```

Fish, Fauna & Behaviour

Fish are procedurally generated in three detail tiers and rendered as multi-line ASCII sprites. No two behave identically — each carries a personality vector.

TIER	ROWS	ROLE	EXAMPLE
Micro	1	fast shoaling fry	`><(°>`
Small	2	body + tail	`<:::::°>`
Detailed	3–7	showpiece fish	reef · dart · pike · puffer · **koi** · **angelfish**

ANGELFISH

```
      /\
     /::\___
  __/:::::: \
<=<::::::::: O>
  ‾‾\:::::: /
     \::/‾‾‾
      \/
```

Tall triangular body, long dorsal & anal fins, streamer tail.

KOI

```
        _______
  _.-~ /.°:.:°.\ ~-._
<~~(::::°:.:°::::: 0 >~
  '-~ \.°:.:°./ ~-'
        ‾‾‾‾‾‾‾
```

Long flowing body, barbels, spotted pattern, trailing fins.

BOIDS — HOW THEY SWIM

Movement emerges from classic steering behaviours, blended per-fish by personality:

`cohesion` `alignment` `separation` `wander` `edge-avoid` `predator-flee` `food-seek`

TRAIT	EFFECT
speed / energy	cruise speed and how hard it darts when startled
schooling	how strongly it joins its species' shoal
curiosity	attraction toward sonar / new events
depth band	preferred vertical zone in the water column

AMBIENT FAUNA

`jellyfish` `sea turtle` `octopus` `stingray` `whale` `shrimp` — lower-density characters that drift through for variety.

What the Tank Is Telling You

This is the heart of Aquarium Mode: every behaviour maps to a real signal. Learn this table and you can read endpoint health at a glance.

SIGNAL	IN THE AQUARIUM
Low load, low risk	HEALTHY — calm violet water, relaxed schooling, soft bubbles.
High CPU	BUSY — livelier fish, stronger current.
Risk score crosses threshold	THREAT — water darkens & reddens, a **predator** appears, schools scatter, glitch burst.
Network spike	**Sonar sweep** expands; curious fish investigate.
Periodic inspection	**AI analysis beam** scans a “suspicious” fish, then it returns to normal.

THE PREDATOR (RANSOMWARE)

```
           ___
      ,--''   `.
 ___/  o    `--.__
(____  ><  ____/>
     \__    _.-'
        `--'
```

Hunts the nearest fish and sweeps across the tank, then leaves. Its presence is what tips the water into the threat state.

SONAR & AI BEAM

Sonar — an expanding cyan ring fired on network bursts; fish with high curiosity swim toward it.

AI analysis — a dashed beam drops from the surface with a reticle box and the label `AI ANALYSIS`. The inspected fish pulses emerald, then resumes normal behaviour — “analyzed & cleared.”

DESIGN PRINCIPLE

Educational, not alarming. Bright red is reserved **only** for an active threat — everything else lives in the calm violet/cyan/emerald palette.

The Dashboard Overlay

A semi-transparent HUD floats over the tank. It is intentionally minimal — status at a glance, detail on demand.

ELEMENT	WHERE	SHOWS
Brand	top-left	**NYTHOS · AQUARIUM MODE**
Status	top-right	● SECURE / ● ACTIVE / ▲ THREAT DETECTED
Scores	top-right	Health & Risk (0–100)
Telemetry	top-right	CPU % · MEM % · NET KB/s
Debug overlay	bottom-left	FPS, entity/particle counts, RSS memory, process count (type `debug`)

HOW TELEMETRY DRIVES THE TANK

The `telemetry` module polls real metrics with `psutil` on a background thread and maps them onto the scene:

METRIC	DRIVES
CPU load	fish liveliness + current strength; the BUSY state
Memory / process pressure	feeds the composite **risk score**
Network throughput	spikes trigger sonar sweeps
Risk threshold	flips the scene into THREAT (predator + scatter)

INTEGRATION SEAM

Replace `Telemetry._sample()` with the real Nythos agent feed (threat detections, open ports, service/app inventory, endpoint health score) and every visual reacts automatically. The aquarium is data-driven — never random — whenever live data exists.

Driving the Aquarium

RUN MODES

MODE	USE
`windowed`	dev / background dashboard
`fullscreen`	splash + full experience
`screensaver`	exits on any input
`kiosk`	conference / demo machine

CONTROLS

`Esc` exit `F11 / F` toggle fullscreen
`F2` windowed dashboard `H` HUD `M` audio
`T` demo threat `S` sonar `W` whale
`+ / −` add / remove a fish

EASTER EGGS

TRIGGER	EFFECT
Konami code	fish become dragons
type `fredo`	a scuba diver with a glowing laptop appears
type `mythos`	a giant phoenix silhouette beneath the water
type `debug`	FPS / entity counts / telemetry / memory overlay

08 — INSTALL

Install as a Program

```
> pwsh -ExecutionPolicy Bypass -File build_exe.ps1       # → dist\NythosAquarium.exe
> pwsh -ExecutionPolicy Bypass -File install_aquarium.ps1 # shortcuts + uninstall entry
```

Installs per-user to `%LOCALAPPDATA%\Programs\NythosAquarium` with Start Menu & Desktop shortcuts and an entry under **Settings › Apps**. Or run from source: `python -m aquarium --mode fullscreen`.

How to Properly Assess It

Use this as a repeatable evaluation pass. Run in `fullscreen`, let it settle ~30 s, then type `debug` to reveal the live stats overlay before scoring.

A • VISUAL QUALITY

- [ ] Depth gradient + light rays read as deep ocean
- [ ] Fish are recognisable, varied, not copy-pasted
- [ ] Reef, treasure & kelp anchor the floor
- [ ] Palette stays violet/cyan; red only on threat
- [ ] No flicker, tearing, or popping at edges

B • BEHAVIOURAL REALISM

- [ ] Fish school, separate & wander organically
- [ ] No two fish move identically
- [ ] Schools scatter from the predator (press `T`)
- [ ] Curious fish drift toward sonar (press `S`)
- [ ] Fish stay in the water column, above the reef

C • SECURITY MAPPING

- [ ] Status matches load (SECURE / ACTIVE / THREAT)
- [ ] Health & Risk track CPU/MEM in the HUD
- [ ] Threat demo darkens water + spawns predator
- [ ] AI beam appears periodically & clears a fish
- [ ] Network activity triggers a sonar sweep

D • PERFORMANCE & STABILITY

- [ ] **FPS** ≥ 30 windowed (target 60); steady, no stutter
- [ ] **RSS memory** flat over time (no leak)
- [ ] CPU reasonable while idle
- [ ] Survives resize, F11/F2 toggling, long run
- [ ] Splash always completes in ~4–5 s

MEASURING PERFORMANCE

The `debug` overlay (bottom-left) reports **FPS**, fish / fauna / particle / effect counts, and process **RSS** memory. Watch RSS for a minute — it should hold steady. If FPS sags on a very large tank, lower the fish count (`–`) or background quality; the engine is built to degrade gracefully.

SCORING RUBRIC

BAND	VISUAL	BEHAVIOUR	MAPPING	PERF
Excellent	cinematic, cohesive	clearly organic	all signals correct	60 FPS, flat RSS
Good	polished, minor nits	believable	states correct	≥40 FPS, stable
Needs work	flat / repetitive	robotic	signals miswired	<30 FPS or leak

> KNOWN TRADE-OFF
>
> Tkinter Canvas text is CPU-bound; a very large tank at 4K can dip below 60 FPS. That is the boundary the optional OpenGL backend (roadmap) is designed to cross. At the default fish count it runs smoothly.